

Министерство образования Республики Беларусь

Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Факультет: Компьютерных систем и сетей

Кафедра: Электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЁТ
к лабораторной работе № 1
на тему
ИНТЕГРИРОВАННАЯ СРЕДА РАЗРАБОТКИ ПО.
КАЛЬКУЛЯТОР ГРОМКОСТИ ЖИЗНИ»
Вариант 30

Студент:

И.М. Серков

Преподаватель:

Д.В. Куприянова

Минск 2026

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить возможности Microsoft Visual Studio Code при создании объектно-ориентированного приложения на языке C++, освоить сборку проекта с помощью CMake и отладку программы с графическим интерфейсом WinAPI, а также применить архитектурный паттерн MVC с активной моделью.

В соответствии с вариантом 30 необходимо разработать калькулятор громкости жизни. Для выполнения работы поставлены следующие задачи:

1. Создать проект C++ в Microsoft Visual Studio Code, настроить сборку CMake, запуск и отладку;
2. Описать средства работы с UML и SysML v2 и представить структуру приложения;
3. Разработать главное окно WinAPI с кнопкой «Ввести данные» и дополнительное окно для ввода количества разговоров, прослушиваний музыки и случаев ругани за день;
4. Обеспечить сохранение последних корректных значений и их восстановление при повторном открытии окна ввода;
5. Реализовать оценку уровня шума жизни, рекомендацию «беруши» или «караоке» и сообщения о некорректном вводе;
6. Разделить GUI и предметную логику по MVC, реализовать активную модель; объявления классов разместить в .hpp, реализации – в .cpp;
7. Проверить модель и контроллер, описать сценарии проверки интерфейса и подготовить ответы на контрольные вопросы.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Интегрированная среда разработки и её компоненты

Интегрированная среда разработки объединяет редактор, средства навигации, сборки, запуска и отладки. Microsoft Visual Studio Code является расширяемым редактором: для разработки на C++ используются расширения C/C++ и CMake Tools. Компилятор и Windows SDK устанавливаются отдельно. CMake описывает состав проекта и формирует файлы для выбранной системы сборки.

2.2 Редактирование и навигация в Microsoft Visual Studio Code

Редактор поддерживает подсветку синтаксиса, автодополнение, форматирование, переход к объявлению и поиск использований. В стандартной раскладке Windows F12 открывает определение, Shift+F12 – ссылки, F2 – переименование, Shift+Alt+F – форматирование. Файлы проекта доступны в Explorer, диагностика – в Problems, команды сборки – через палитру команд и CMake Tools.

2.3 Сборка, запуск и отладка C++-приложения

Файлы .cpp компилируются в объектные файлы и связываются в исполняемый файл. Заголовки .hpp содержат объявления типов и методов. Команда cmake -S . -B build конфигурирует проект, cmake --build build --config Debug выполняет сборку. В Windows используется MSVC с Windows SDK. В Microsoft Visual Studio Code точка останова устанавливается клавишей F9; средства Run and Debug позволяют наблюдать переменные и стек вызовов, выполнять шаги с обходом и заходом в функцию.

2.4 Паттерн MVC

MVC разделяет приложение на три части. Model хранит состояние и выполняет предметную логику. View формирует интерфейс и отображает данные. Controller обрабатывает действия пользователя и передаёт команды модели. Такое разделение позволяет проверять расчет и обработку ввода независимо от WinAPI. Модель и контроллер не включают windows.h и не создают элементы интерфейса.

2.5 Активная и пассивная модель

В варианте с пассивной моделью контроллер после изменения данных инициирует обновление представления. Активная модель сама уведомляет наблюдателя об изменении состояния. В приложении LifeModel вызывает метод onModelChanged интерфейса ModelObserver.

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Постановка задачи

Разработан каркас калькулятора громкости жизни. Пользователь вводит, сколько раз за день он разговаривает, слушает музыку и ругается. Главная форма содержит кнопку «Ввести данные», открывающую модальное окно InputDialog. После успешного ввода отображаются все три величины, условный уровень шума и рекомендация. Последние корректные данные остаются в состоянии приложения и восстанавливаются при повторном вводе.

3.2 Настройка среды и средства UML и SysML v2

Проект LifeVolume открывается в Microsoft Visual Studio Code. Для C++ предусмотрены расширения C/C++ и CMake Tools, компилятор MSVC и Windows SDK. В CMakeLists.txt заданы C++17, отдельная библиотека life_core и графическая цель LifeVolume. Для UML подготовлен файл docs/mvc.puml в текстовой нотации PlantUML. Его можно просматривать с совместимым расширением. Для SysML v2 ниже приведено структурное представление; наличие установленного расширения и запуск среды в рамках автоматической проверки не подтверждались.

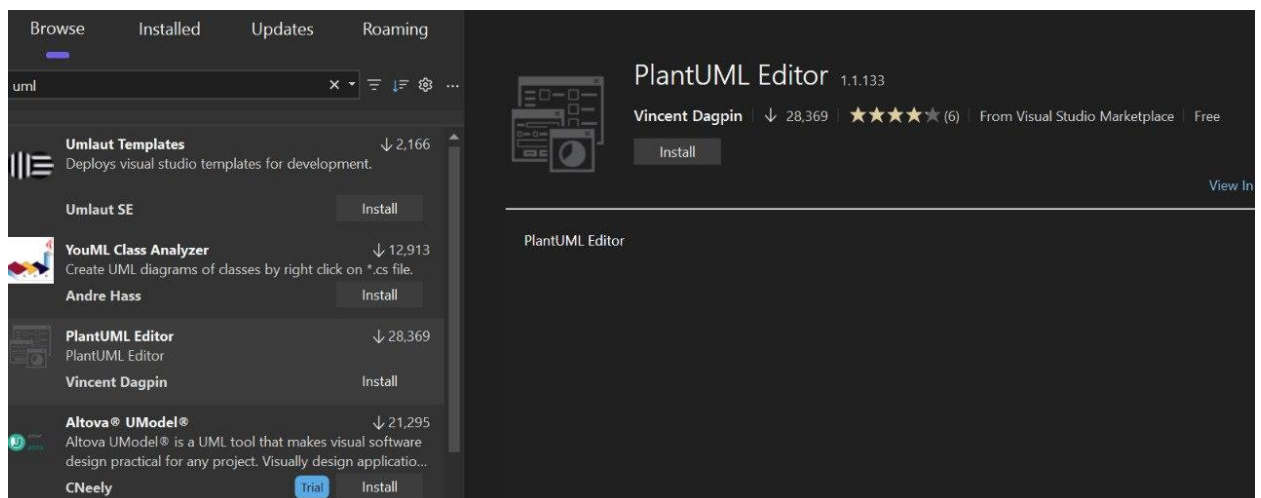


Рисунок 3.1 – Средства для работы с UML

Структурное представление системы в нотации, близкой к SysML v2 (эскиз состава):

```
package LifeVolumeCalculator {  
    part def LifeModel;  
    part def MainView;  
    part def InputDialog;  
    part def LifeController;  
    part application {  
        part model : LifeModel;
```

```

part view : MainView;
part dialog : InputDialog;
part controller : LifeController;
// View передает действия контроллеру;
// Controller изменяет модель;
// Model уведомляет View.
}
}

```

3.3 Сценарий демонстрации отладчика

Для демонстрации следует собрать цель LifeVolume в конфигурации Debug и запустить её через CMake: Debug. В LifeModel.cpp точка останова ставится в методе setData на вычислении next.score. При вводе 20, 9, 4 после выполнения строки ожидается next.score = 50, затем next.earplugs = true. После присваивания state_ = next модель вызывает onModelChanged. На рисунке показан сценарий наблюдения переменных, а не снимок выполненного сеанса отладки.

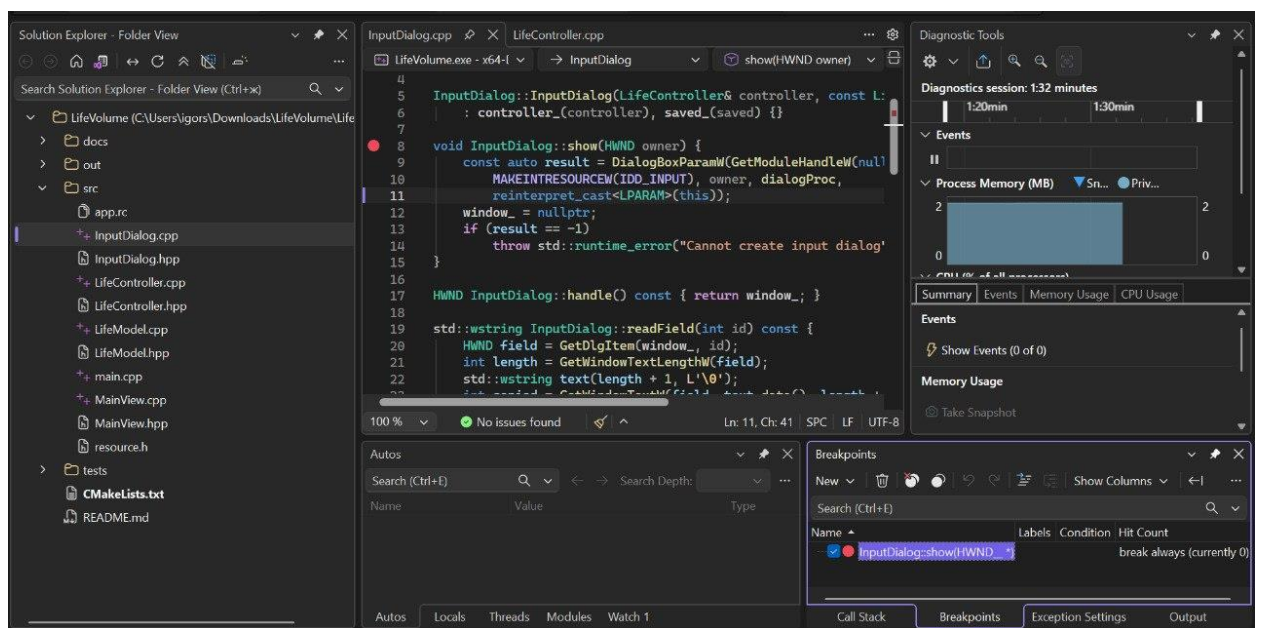


Рисунок 3.2 – Остановка программы в отладчике

3.4 Структура проекта и каркас MVC с активной моделью

Проект разделён на следующие компоненты:

- LifeModel.hpp и LifeModel.cpp – активная модель, структуры LifeData и LifeState, проверка диапазонов и расчет условного индекса;
- MainView.hpp и MainView.cpp – главное окно WinAPI, реализация LifeView и ModelObserver, хранение копии отображаемого состояния;
- InputDialog.hpp и InputDialog.cpp – модальное окно ввода; app.rc содержит его ресурс, resource.h – идентификаторы элементов;
- LifeController.hpp и LifeController.cpp – обработка команд, проверка текстовых значений и передача корректных чисел модели;
- main.cpp – точка входа wWinMain и цикл сообщений; CMakeLists.txt – сборка, tests/core_tests.cpp – независимые проверки модели и контроллера.

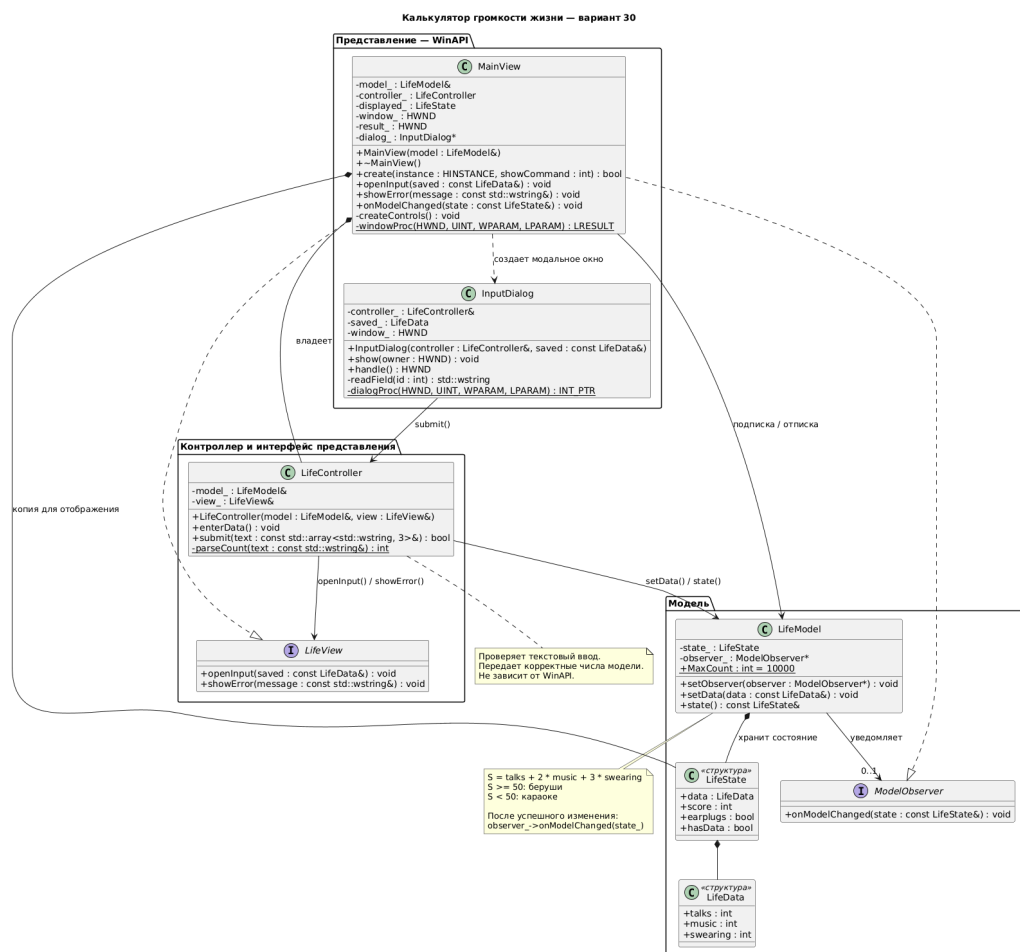


Рисунок 3.3 – Структура MVC и направление уведомлений

3.5 Модель

LifeModel реализует активную модель. Метод setData принимает три целых числа от 0 до 10000 включительно. Поскольку условие не задает формулу, принято учебное правило $S = R + 2M + 3F$, где R – разговоры, M –

прослушивания музыки, F – случаи ругани. При $S \geq 50$ рекомендация – «беруши», иначе – «караоке». Это условные баллы, а не децибелы. Максимум равен 60000. Новое состояние фиксируется только после проверки, затем вызывается onModelChanged.

3.6 Представление

MainView создает главное окно через CreateWindowExW и отображает три исходных значения, индекс и рекомендацию. Метод onModelChanged сохраняет копию LifeState и обновляет текстовые метки. InputDialog открывается через DialogBoxParamW, использует стандартные поля EDIT и кнопки «Рассчитать» и «Отмена». Оно извлекает текст и передает его контроллеру. Формулы, пороги и числовой разбор в GUI отсутствуют.

3.7 Контроллер

LifeController получает команду enterData и передает сохраненные значения в LifeView::openInput. Метод submit разбирает строки в числа, отклоняет пустые поля, знаки, пробелы, дроби, посторонние символы и значения больше 10000. Проверка верхней границы выполняется до умножения, предотвращая переполнение. При ошибке контроллер вызывает LifeView::showError; конкретное окно MessageBoxW создает представление. При успехе контроллер вызывает LifeModel::setData.

3.8 Запоминание данных и обработка ошибок

Последнее корректное состояние хранится в модели и в копии MainView. Перед каждым открытием InputDialog контроллер получает его из модели. При первом вводе поля заполнены нулями. Ошибка не меняет сохраненные данные и не закрывает окно, что позволяет исправить ввод. Отмена и закрытие дополнительного окна также не изменяют результат. Сохранение действует в пределах текущего запуска приложения; запись на диск не требуется условием и не реализована.

4 РЕЗУЛЬТАТЫ РАБОТЫ

Подготовлен проект варианта 30 на C++17 с WinAPI, CMake и активной моделью MVC. Модель и контроллер собраны и проверены через CTest в Linux: пройдены проверки нулей, расчета $10/5/2 \rightarrow 26$, границы 49/50, максимума 60000, уведомлений и восстановления состояния. Отклонены 27 комбинаций некорректного текстового ввода и три недопустимых набора чисел. Графическая часть требует проверки в Windows. Рисунки 4.1–4.3 являются схемами форм; они не подтверждают фактический запуск GUI.

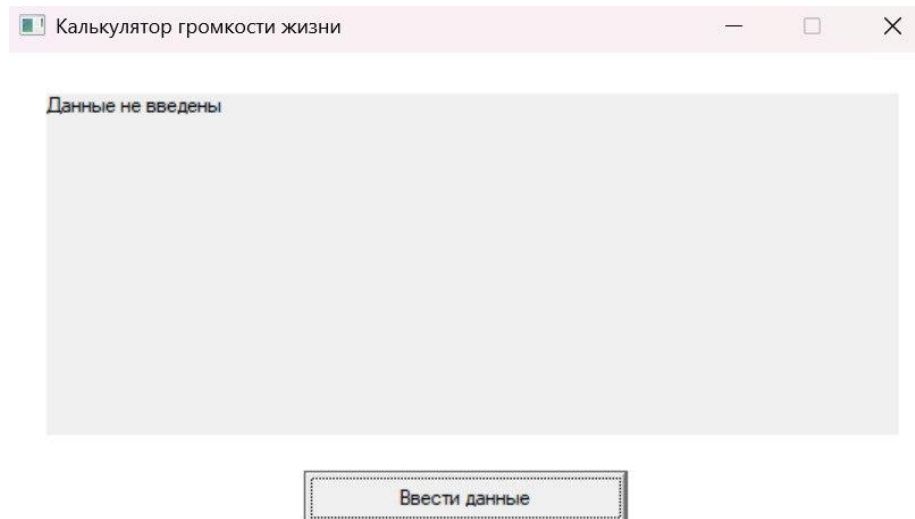


Рисунок 4.1 – Схема главной формы до ввода

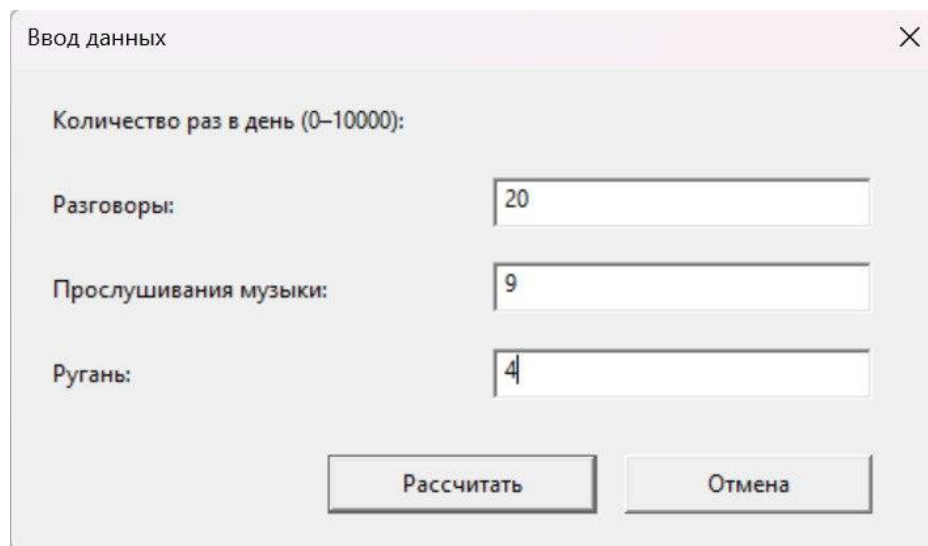


Рисунок 4.2 – Схема дополнительного окна ввода

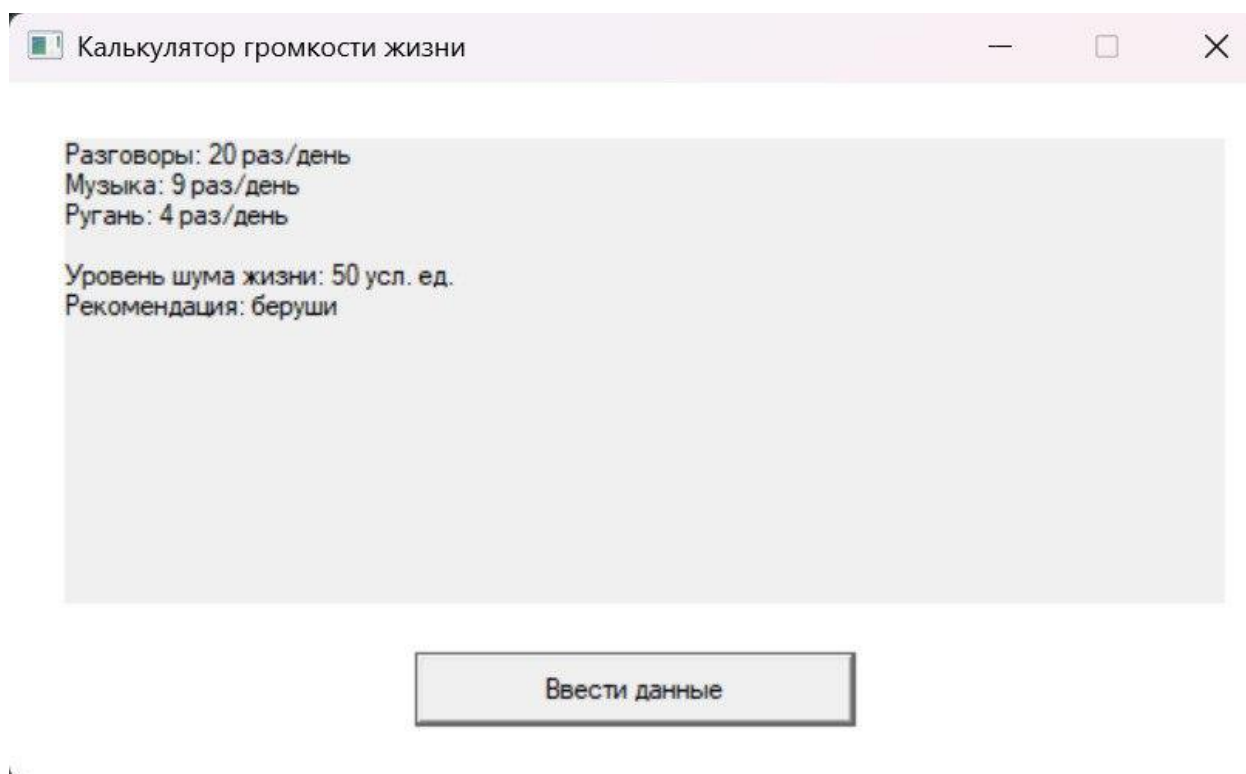


Рисунок 4.3 – Схема результата для данных 20, 9, 4

5 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1 Какие компоненты входят в состав IDE?

В состав IDE входят редактор кода, средства навигации, компилятор, система сборки и запуска, отладчик, терминал и интеграция с системой контроля версий. Microsoft Visual Studio Code получает функции среды C++ через расширения C/C++ и CMake Tools, используя отдельно установленный компилятор и Windows SDK.

Вопрос 2 Каковы основные приёмы работы с редактором кода?

К основным приёмам относятся автодополнение, быстрые исправления, автоматический импорт, форматирование, переход к объявлению, поиск использований, переименование, просмотр иерархии классов и анализ предупреждений. Эти операции позволяют изменять код быстрее и уменьшают вероятность механических ошибок.

Вопрос 3 Каким образом выполняется сборка и запуск проекта?

Проект конфигурируется командой `cmake -S . -B build` и собирается командой `cmake --build build --config Debug`. При генераторе Visual Studio исполняемый файл находится в `build/Debug/LifeVolume.exe`. В VS Code доступны команды CMake: Configure, CMake: Build и CMake: Debug. Автоматические проверки запускаются командой `ctest --test-dir build -C Debug --output-on-failure`.

Вопрос 4 Что такое MVC?

MVC – архитектурный паттерн, разделяющий данные и предметную логику, графический интерфейс и обработку действий пользователя. В данной работе модель – LifeModel, представление – MainView и InputDialog, контроллер – LifeController. GUI передает события контроллеру, но не рассчитывает уровень шума.

Вопрос 5 Чем отличается активная модель от пассивной?

Пассивная модель не уведомляет представление, поэтому его обновлением управляет контроллер. Активная модель самостоятельно сообщает об изменении состояния. В данной работе LifeModel вызывает `ModelObserver::onModelChanged` после успешного сохранения LifeState, а MainView обновляет элементы формы. При неверном вводе состояние и отображенный результат сохраняются.

ЗАКЛЮЧЕНИЕ

В ходе работы подготовлен проект для Microsoft Visual Studio Code на языке C++17 со сборкой CMake. Описаны редактирование, сборка, запуск и сценарий отладки, рассмотрены UML и структурное представление SysML v2. Объявления и реализации классов разделены на файлы .hpp и .cpp.

Для варианта 30 разработан каркас калькулятора громкости жизни с интерфейсом WinAPI. Расчет, проверка ввода и отображение разделены по MVC. Активная модель уведомляет главное окно об изменении данных; предусмотрены восстановление значений и обработка ошибок. Автоматические проверки модели и контроллера пройдены. Демонстрация и ручная проверка графической части выполняются в Windows.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода приложения

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(LifeVolume LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
add_library(life_core src/LifeModel.cpp src/LifeController.cpp)
target_include_directories(life_core PUBLIC src)
if(MSVC)
    target_compile_options(life_core PUBLIC /utf-8 /W4)
else()
    target_compile_options(life_core PUBLIC -Wall -Wextra -Wpedantic)
endif()
if(WIN32)
    enable_language(RC)
    add_executable(LifeVolume WIN32 src/main.cpp src/MainView.cpp
        src/InputDialog.cpp src/app.rc)
    target_compile_definitions(LifeVolume PRIVATE UNICODE _UNICODE NOMINMAX)
    target_link_libraries(LifeVolume PRIVATE life_core user32 gdi32)
    if(MINGW)
        target_link_options(LifeVolume PRIVATE -municode)
    endif()
endif()
include(CTest)
if(BUILD_TESTING)
    add_executable(core_tests tests/core_tests.cpp)
    target_link_libraries(core_tests PRIVATE life_core)
    add_test(NAME core_tests COMMAND core_tests)
endif()
```

src/LifeModel.hpp

```
#pragma once

struct LifeData {
    int talks = 0;
    int music = 0;
    int swearing = 0;
};

struct LifeState {
    LifeData data;
    int score = 0;
    bool earplugs = false;
    bool hasData = false;
};

class ModelObserver {
public:
    virtual ~ModelObserver() = default;
    virtual void onModelChanged(const LifeState& state) = 0;
};

class LifeModel {
public:
    static constexpr int MaxCount = 10000;
    void setObserver(ModelObserver* observer);
    void setData(const LifeData& data);
    const LifeState& state() const;
private:
    LifeState state_;
    ModelObserver* observer_ = nullptr;
};
```

src/LifeModel.cpp

```
#include "LifeModel.hpp"
#include <stdexcept>

void LifeModel::setObserver(ModelObserver* observer) {
    observer_ = observer;
}

const LifeState& LifeModel::state() const {
    return state_;
}

void LifeModel::setData(const LifeData& data) {
    for (int value : {data.talks, data.music, data.swearing}) {
        if (value < 0 || value > MaxCount)
            throw std::invalid_argument("Count outside 0..10000");
    }
    LifeState next;
    next.data = data;
    next.score = data.talks + 2 * data.music + 3 * data.swearing;
    next.earplugs = next.score >= 50;
    next.hasData = true;
    state_ = next;
    if (observer_)
        observer_>onModelChanged(state_);
}
```

src/LifeController.hpp

```
#pragma once
#include "LifeModel.hpp"
#include <array>
#include <string>

class LifeView {
public:
    virtual ~LifeView() = default;
    virtual void openInput(const LifeData& saved) = 0;
    virtual void showError(const std::wstring& message) = 0;
};

class LifeController {
public:
    LifeController(LifeModel& model, LifeView& view);
    void enterData();
    bool submit(const std::array<std::wstring, 3>& text);
private:
    static int parseCount(const std::wstring& text);
    LifeModel& model_;
    LifeView& view_;
};
```

src/LifeController.cpp

```
#include "LifeController.hpp"
#include <stdexcept>

LifeController::LifeController(LifeModel& model, LifeView& view)
    : model_(model), view_(view) {}

void LifeController::enterData() {
    view_.openInput(model_.state().data);
}

int LifeController::parseCount(const std::wstring& text) {
    if (text.empty())
        throw std::invalid_argument("Empty input");
    int value = 0;
    for (wchar_t c : text) {
        if (c < L'0' || c > L'9')
```

```

        throw std::invalid_argument("Expected digits");
        const int digit = c - L'0';
        if (value > (LifeModel::MaxCount - digit) / 10)
            throw std::invalid_argument("Count too large");
        value = value * 10 + digit;
    }
    return value;
}

bool LifeController::submit(const std::array<std::wstring, 3>& text) {
    LifeData data;
    try {
        data = {parseCount(text[0]), parseCount(text[1]),
                parseCount(text[2])};
    } catch (const std::invalid_argument&) {
        view_.showError(L"Введите в каждое поле целое число от 0 до 10000. "
                        L"Пробелы, знаки и дроби недопустимы.");
        return false;
    }
    model_.setData(data);
    return true;
}

```

src/MainView.hpp

```

#pragma once
#include <windows.h>
#include "LifeController.hpp"
class InputDialog;

class MainView final : public LifeView, public ModelObserver {
public:
    explicit MainView(LifeModel& model);
    ~MainView() override;
    bool create(HINSTANCE instance, int showCommand);
    void openInput(const LifeData& saved) override;
    void showError(const std::wstring& message) override;
    void onModelChanged(const LifeState& state) override;
private:
    static LRESULT CALLBACK windowProc(HWND, UINT, WPARAM, LPARAM);
    void createControls();
    LifeModel& model_;
    LifeController controller_;
    LifeState displayed_;
    HWND window_ = nullptr;
    HWND result_ = nullptr;
    InputDialog* dialog_ = nullptr;
};

```

src/MainView.cpp

```

#include "MainView.hpp"
#include "InputDialog.hpp"
#include "resource.h"
#include <stdexcept>

MainView::MainView(LifeModel& model)
    : model_(model), controller_(model, *this) {
    model_.setObserver(this);
}

MainView::~MainView() { model_.setObserver(nullptr); }

bool MainView::create(HINSTANCE instance, int showCommand) {
    WNDCLASSW wc{};
    wc.lpfnWndProc = windowProc;
    wc.hInstance = instance;
    wc.lpszClassName = L"LifeVolumeMain";
    wc.hCursor = LoadCursorW(nullptr, IDC_ARROW);
    wc.hbrBackground = reinterpret_cast<HBRUSH>(COLOR_WINDOW + 1);
    if (!RegisterClassW(&wc)) return false;
}

```

```

        window_ = CreateWindowExW(0, wc.lpszClassName,
            L"Калькулятор громкости жизни", WS_OVERLAPPED | WS_CAPTION |
            WS_SYSMENU | WS_MINIMIZEBOX, CW_USEDEFAULT, CW_USEDEFAULT,
            560, 340, nullptr, nullptr, instance, this);
        if (!window_) return false;
        ShowWindow(window_, showCommand);
        return true;
    }

void MainView::createControls() {
    auto instance = GetModuleHandleW(nullptr);
    result_ = CreateWindowExW(0, L"STATIC", L"Данные не введены",
        WS_CHILD | WS_VISIBLE, 24, 24, 500, 200,
        window_, nullptr, instance, nullptr);
    HWND button = CreateWindowExW(0, L"BUTTON", L"Ввести данные",
        WS_CHILD | WS_VISIBLE | WS_TABSTOP | BS_DEFPUSHBUTTON,
        175, 245, 190, 32, window_,
        reinterpret_cast<HMENU>(IDC_ENTER), instance, nullptr);
    for (HWND control : {result_, button})
        SendMessageW(control, WM_SETFONT,
            reinterpret_cast<WPARAM>(GetStockObject(DEFAULT_GUI_FONT)), TRUE);
    SetFocus(button);
}

void MainView::onModelChanged(const LifeState& state) {
    displayed_ = state;
    const auto& d = displayed_.data;
    std::wstring text = L"Разговоры: " + std::to_wstring(d.talks)
        + L" раз/день\r\nМузыка: " + std::to_wstring(d.music)
        + L" раз/день\r\nПугань: " + std::to_wstring(d.swearing)
        + L" раз/день\r\n\r\nУровень шума жизни: "
        + std::to_wstring(displayed_.score) + L" усл. ед.\r\nРекомендация: "
        + (displayed_.earplugs ? L"беруши" : L"караоке");
    SetWindowTextW(result_, text.c_str());
}

void MainView::openInput(const LifeData& saved) {
    InputDialog dialog(controller_, saved);
    dialog_ = &dialog;
    try {
        dialog.show(window_);
    } catch (const std::runtime_error&) {
        dialog_ = nullptr;
        showError(L"Не удалось открыть окно ввода.");
    }
    dialog_ = nullptr;
}

void MainView::showError(const std::wstring& message) {
    HWND owner = dialog_ ? dialog_>handle() : window_;
    MessageBoxW(owner, message.c_str(), L"Ошибка", MB_OK | MB_ICONERROR);
}

LRESULT CALLBACK MainView::windowProc(
    HWND hwnd, UINT msg, WPARAM wp, LPARAM lp) {
    auto* self = reinterpret_cast<MainView*>(
        GetWindowLongPtrW(hwnd, GWLP_USERDATA));
    if (msg == WM_NCCREATE) {
        auto* cs = reinterpret_cast<CREATESTRUCTW*>(lp);
        self = static_cast<MainView*>(cs->lpCreateParams);
        self->window_ = hwnd;
        SetWindowLongPtrW(hwnd, GWLP_USERDATA,
            reinterpret_cast<LONG_PTR>(self));
    }
    if (self) {
        if (msg == WM_CREATE) { self->createControls(); return 0; }
        if (msg == WM_COMMAND && LOWORD(wp) == IDC_ENTER) {
            self->controller_.enterData();
            return 0;
        }
    }
}

```

```

        if (msg == WM_DESTROY) { PostQuitMessage(0); return 0; }
    }
    return DefWindowProcW(hwnd, msg, wp, lp);
}

```

src/InputDialog.hpp

```

#pragma once
#include <windows.h>
#include "LifeController.hpp"

class InputDialog {
public:
    InputDialog(LifeController& controller, const LifeData& saved);
    void show(HWND owner);
    HWND handle() const;
private:
    static INT_PTR CALLBACK dialogProc(HWND, UINT, WPARAM, LPARAM);
    std::wstring readField(int id) const;
    LifeController& controller_;
    LifeData saved_;
    HWND window_ = nullptr;
};

```

src/InputDialog.cpp

```

#include "InputDialog.hpp"
#include "resource.h"
#include <stdexcept>

InputDialog::InputDialog(LifeController& controller, const LifeData& saved)
    : controller_(controller), saved_(saved) {}

void InputDialog::show(HWND owner) {
    const auto result = DialogBoxParamW(GetModuleHandleW(nullptr),
        MAKEINTRESOURCEW(IDD_INPUT), owner, dialogProc,
        reinterpret_cast<LPARAM>(this));
    window_ = nullptr;
    if (result == -1)
        throw std::runtime_error("Cannot create input dialog");
}

HWND InputDialog::handle() const { return window_; }

std::wstring InputDialog::readField(int id) const {
    HWND field = GetDlgItem(window_, id);
    int length = GetWindowTextLengthW(field);
    std::wstring text(length + 1, L'\0');
    int copied = GetWindowTextW(field, text.data(), length + 1);
    text.resize(copied);
    return text;
}

INT_PTR CALLBACK InputDialog::dialogProc(
    HWND hwnd, UINT msg, WPARAM wp, LPARAM lp) {
    auto* self = reinterpret_cast<InputDialog*>(
        GetWindowLongPtrW(hwnd, DWLP_USER));
    if (msg == WM_INITDIALOG) {
        self = reinterpret_cast<InputDialog*>(lp);
        self->window_ = hwnd;
        SetWindowLongPtrW(hwnd, DWLP_USER, lp);
        SetDlgItemInt(hwnd, IDC_TALKS, self->saved_.talks, FALSE);
        SetDlgItemInt(hwnd, IDC_MUSIC, self->saved_.music, FALSE);
        SetDlgItemInt(hwnd, IDC_SWEARING, self->saved_.swearing, FALSE);
        return TRUE;
    }
    if (!self) return FALSE;
    if (msg == WM_COMMAND && LOWORD(wp) == IDOK) {
        if (self->controller_.submit({self->readField(IDC_TALKS),
            self->readField(IDC_MUSIC), self->readField(IDC_SWEARING)}))
            EndDialog(hwnd, IDOK);
    }
}

```

```

        return TRUE;
    }
    if ((msg == WM_COMMAND && LOWORD(wp) == IDCANCEL) || msg == WM_CLOSE) {
        EndDialog(hwnd, IDCANCEL);
        return TRUE;
    }
    return FALSE;
}

```

src/resource.h

```

#pragma once
#define IDD_INPUT 101
#define IDC_TALKS 1001
#define IDC_MUSIC 1002
#define IDC_SWEARING 1003
#define IDC_ENTER 1004

```

src/app.rc

```

#pragma code_page(65001)
#include <windows.h>
#include "resource.h"
IDD_INPUT DIALOGEX 0, 0, 270, 132
STYLE DS_SETFONT | DS_MODALFRAME | WS_POPUP | WS_CAPTION | WS_SYSMENU
CAPTION "Ввод данных"
FONT 9, "Segoe UI"
BEGIN
    LTEXT "Количество раз в день (0-10000):", -1, 12, 10, 240, 12
    LTEXT "Разговоры:", -1, 12, 33, 110, 12
    EDITTEXT IDC_TALKS, 140, 30, 110, 14, ES_AUTOHSCROLL | WS_TABSTOP
    LTEXT "Прослушивания музыки:", -1, 12, 56, 125, 12
    EDITTEXT IDC_MUSIC, 140, 53, 110, 14, ES_AUTOHSCROLL | WS_TABSTOP
    LTEXT "Ругань:", -1, 12, 79, 110, 12
    EDITTEXT IDC_SWEARING, 140, 76, 110, 14, ES_AUTOHSCROLL | WS_TABSTOP
    DEFPUSHBUTTON "Рассчитать", IDOK, 92, 105, 78, 16
    PUSHBUTTON "Отмена", IDCANCEL, 178, 105, 72, 16
END

```

src/main.cpp

```

#include "MainView.hpp"

int WINAPI wWinMain(HINSTANCE instance, HINSTANCE, PWSTR, int showCommand) {
    LifeModel model;
    MainView view(model);
    if (!view.create(instance, showCommand)) {
        MessageBoxW(nullptr, L"Не удалось создать главное окно.",
            L"Ошибка", MB_OK | MB_ICONERROR);
        return 1;
    }
    MSG message{};
    int status;
    while ((status = GetMessageW(&message, nullptr, 0, 0)) > 0) {
        TranslateMessage(&message);
        DispatchMessageW(&message);
    }
    return status == -1 ? 1 : static_cast<int>(message.wParam);
}

```

tests/core_tests.cpp

```

#include "LifeController.hpp"
#include <iostream>
#include <stdexcept>

void check(bool condition) {
    if (!condition) throw std::runtime_error("Test failed");
}

struct Observer : ModelObserver {
    int notifications = 0;
    LifeState last;
}

```

